Smart Inventory Management Design Outline

1. Project Overview

Name: Smart Inventory Management System
Type: Console-based C# Application
Purpose: To manage products in a small inventory efficiently, allowing addition, removal, stock updates, and viewing product information.

---

2. Project Requirements

1. **Add Product** – Users can add a new product with name, price, and stock quantity. Duplicate products are not allowed.
2. **Update Stock** – Users can increase or decrease stock for existing products. Negative stock is prevented.
3. **View Products** – Users can view all products with their price and stock.
4. **Remove Product** – Users can delete a product from the inventory.
5. **Validation** – Inputs for price and stock must be numeric. Product names are case-insensitive.

---

3. Project Objectives

- To practice **array handling** in C#.
- To implement **control structures** (`if-else`, `switch`) and **loops** (`for`, `while`).
- To demonstrate **modular programming** with methods for each main operation.
- To ensure **robust user input validation**.
- To simulate a **real-world inventory management workflow** in a simple console environment.

---

4. Functional Modules / Methods

Method	Purpose
`AddProduct()`	Adds a new product to the inventory; checks duplicates; validates price and stock.
`UpdateStock()`	Updates stock for an existing product; prevents negative stock; validates input.
`ViewProducts()`	Displays all products with name, price, and stock.
`RemoveProduct()`	Deletes a product from the inventory; shifts arrays to maintain continuity.

---

5. Control Structures

- **`switch` statement:** To handle menu selection.
- **`if-else` statements:** For checking product existence, validating input, and preventing negative stock.

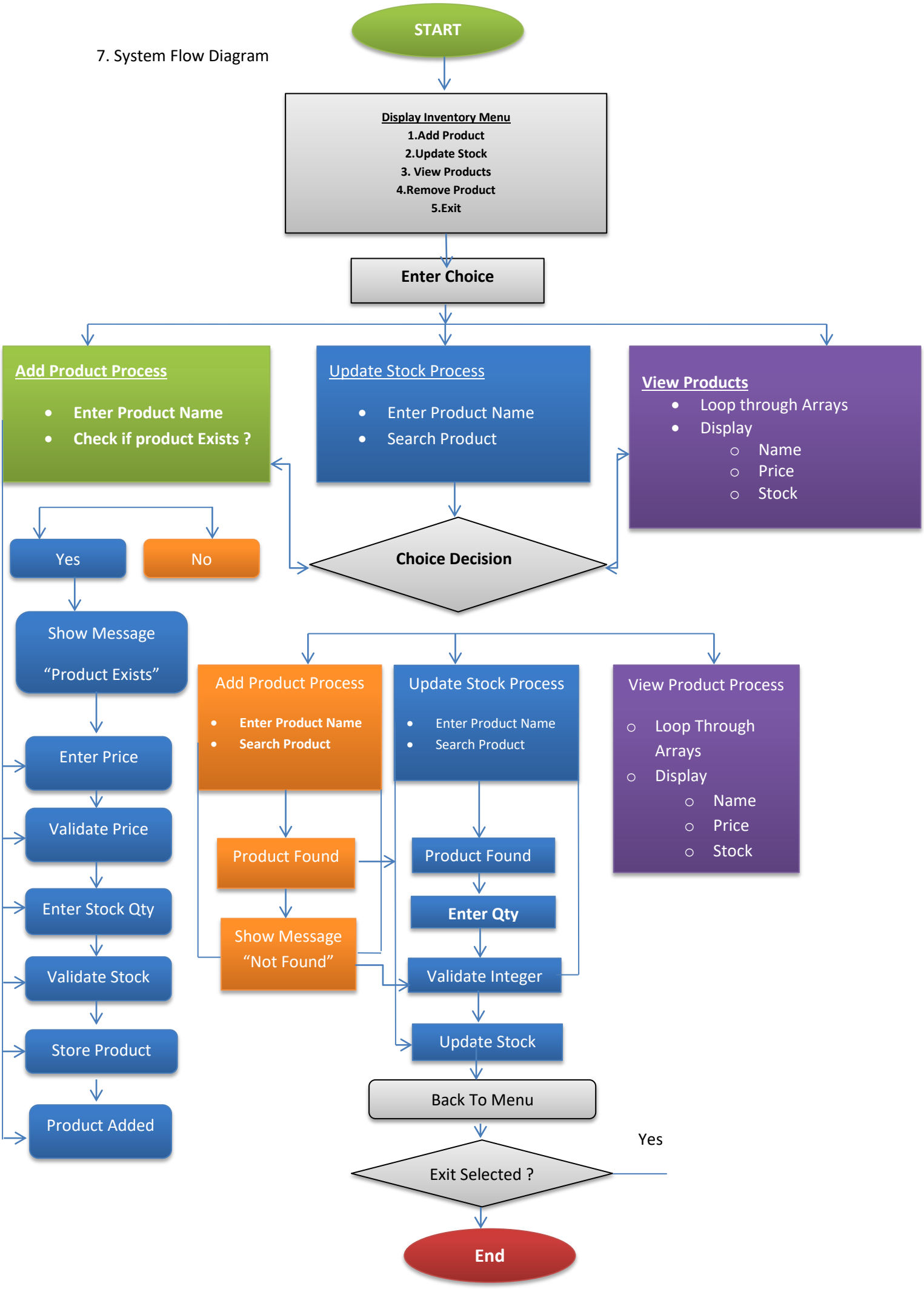
---

6. Loops

- **`while` loops:** For the main menu and input validation.
- **`for` loops:** For searching products in arrays and displaying products.

---

7. System Flow Diagram



8. Array Data Structure

Array	Data Type	Purpose
`productNames[50]`	`string`	Store product names
`productPrices[50]`	`double`	Store product prices
`productStocks[50]`	`int`	Store product stock quantities
`productCount`	`int`	Keep track of number of products

---

9. Input Validation

- **Price:** Must be numeric (`double.TryParse`)
- **Stock:** Must be integer (`int.TryParse`)
- **Stock updates:** Prevent negative values after addition/removal
- **Product Name:** Case-insensitive uniqueness check

This **design outline** covers all points:

- **Requirements & objectives**
- **Design outline**
- **Control structures & loops**
- **Methods**
- **Validation & arrays**